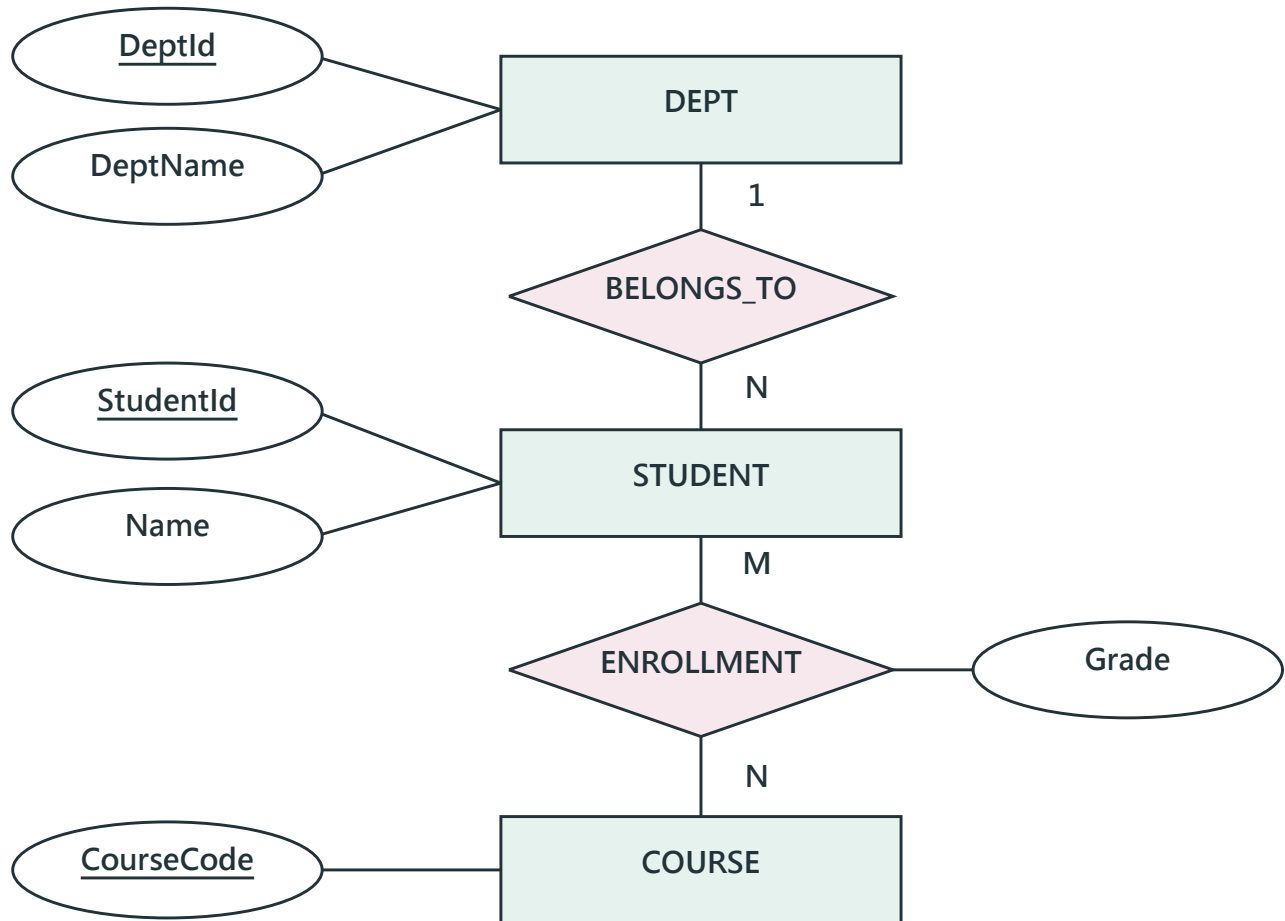


選課案例：學生、系所與課程

教務處記錄同一學期的學生、系所、課程與成績。學生可以尚未分系，也可以還沒選課。先看關係，再看實際資料。



矩形是實體，菱形是關係，橢圓是屬性；底線標出各實體的識別欄位。Grade 接在選課關係上。

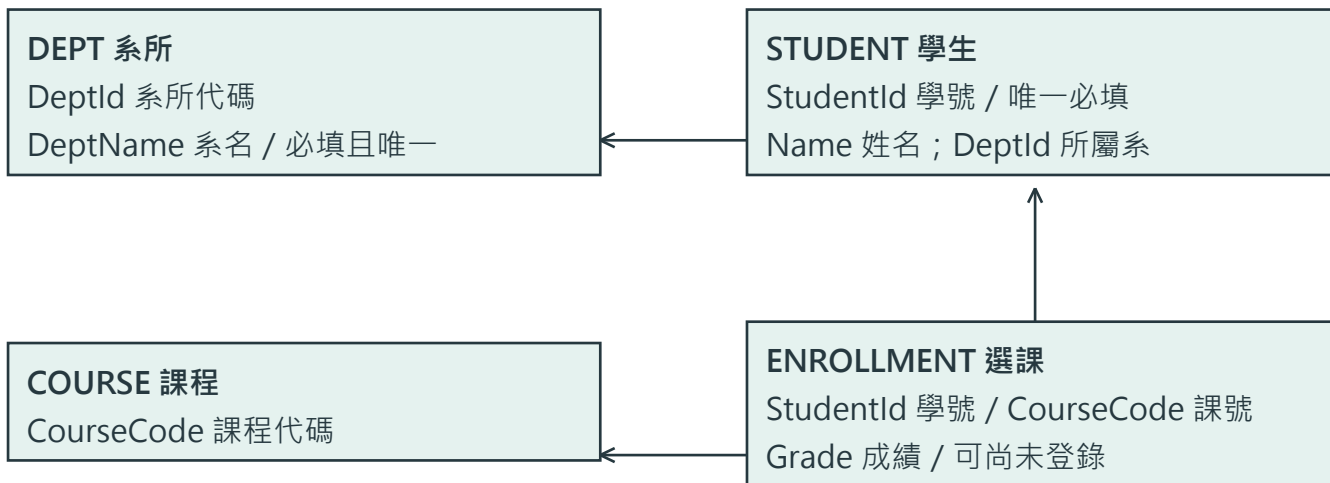
本校先訂好的規則

關係	可以怎麼連？
學生與系所	一位學生屬於 0 或 1 個系；一個系可有 0 到多位學生。
學生與課程	一位學生可修多門課；一門課可有多位學生，也都可以暫無配對。
每筆選課	同一學生同一課程只記一筆成績：0 至 100 的整數，或尚未登錄。

本頁重點：成績屬於「哪位學生修哪門課」，不能只接在學生或課程上。

選課案例：四張表的關聯

沿用上頁的教務處案例。學生與課程各有自己的表；選課另外存學生學號、課程代碼與這個配對的成績。以下都是人工教學資料。



箭頭：依所存代碼，到另一張表查相同代碼的列。

學生可尚未分系。每筆選課必須指向已存在的學生與課程。

每張表如何指出是哪一列？

表	識別欄位	要查到哪張表？
DEPT	DeptId	不參照其他表
STUDENT	StudentId	DeptId 查 DEPT；可尚未分系
COURSE	CourseCode	不參照其他表
ENROLLMENT	StudentId + CourseCode	StudentId 查 STUDENT；CourseCode 查 COURSE

識別一列的欄位稱主鍵 (PRIMARY KEY)；存另一張表的代碼、要求查得到對象的欄位稱外來鍵 (FOREIGN KEY)。圖中箭頭是查找方向，不是新增資料的先後。

本頁重點：先看整張關聯圖，再用下一頁的完整資料逐列查找。

選課案例：完整範例資料

以下是四張表的全部起始資料。DB101 代表資料庫導論，CS102 代表資料庫實作；本例 COURSE 只存課程代碼。

DEPT：全部系所

DeptId	DeptName
D1	資訊管理
D2	會計

COURSE：全部課程

CourseCode
DB101
CS102

D1 是資訊管理系，D2 是會計系；D2 尚無學生，仍保留系所紀錄。

STUDENT：所有學生

StudentId	Name	DeptId
S1	Alex	D1
S2	Alex	D1
S3	Casey	NULL

ENROLLMENT：所有選課

StudentId	CourseCode	Grade
S1	DB101	80
S1	CS102	NULL
S2	DB101	0

S3 的 DeptId 為 NULL，表示尚未分系；S1 / CS102 的 Grade 為 NULL，表示成績尚未登錄。S2 / DB101 的 0 是已知零分。

問題：只寫「Alex 修 DB101」，能指出唯一一筆選課嗎？

回答：不能。上表的 S1、S2 都叫 Alex，也都修 DB101。必須寫 S1 / DB101 或 S2 / DB101 才能指出是哪筆。

本頁重點：S1 修兩門課，配對不同。查詢都讀這份起始資料；各頁修改獨立重設。

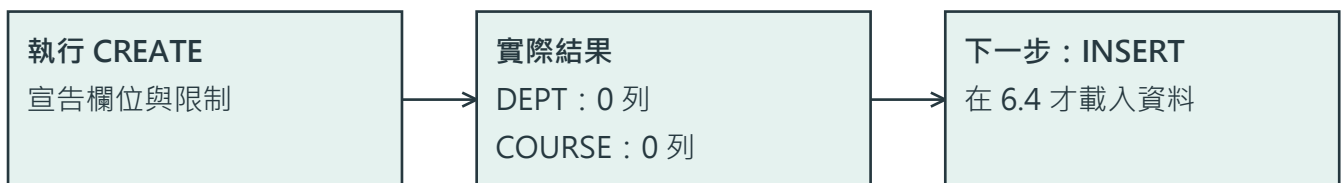
建立系所表與課程表

目標是建立 DEPT、COURSE 的欄位與限制，還不加入 D1 或 DB101。預期：建表成功後，兩表都只有結構，各有 0 列。

D1 與「資訊管理」是文字，欄名後寫 TEXT。不能缺值寫 NOT NULL；識別系所的代碼寫 PRIMARY KEY；系名另須不重複寫 UNIQUE。這些都是建表時宣告的限制 (constraint)。

實作額外補充：SQLite 外來鍵設定。下方兩行 PRAGMA 是 SQLite 專用設定，不是課本的建表語法。第一行開啟外來鍵檢查；最後一行回讀狀態，1 表示開啟、0 表示關閉。

```
PRAGMA foreign_keys = ON;
CREATE TABLE DEPT (
  DeptId TEXT NOT NULL PRIMARY KEY,
  DeptName TEXT NOT NULL UNIQUE
);
CREATE TABLE COURSE (
  CourseCode TEXT NOT NULL PRIMARY KEY
);
PRAGMA foreign_keys;
```



實際：兩表已建立但皆無資料；PRAGMA 回讀 1。本頁尚未宣告外來鍵，這個 1 不是替某欄指定參照。

CREATE TABLE 定義欄位與限制，屬資料定義語言 (DDL)；DEPT 是儲存系所列的 base table。課本用 schema 組織表等物件、catalog 組織 schemas；本例建於 main，不使用課本的 CREATE SCHEMA。

問題：兩表都是 0 列，是否代表建表失敗？

回答：不是。欄位與限制已建立，符合本頁預期；D1、DB101 必須由之後的 INSERT 加入。

本頁重點：答案：CREATE TABLE 已成功建立空表；它不會自動載入案例資料。

欄位的資料型別

先看選課資料的型別對照，再比較「拒絕 80.5」的兩種寫法。

課本型別與本例的用法

要存什麼	課本介紹的型別	本例 SQLite
S1、Alex、D1	CHAR：固定長度；VARCHAR：可變長度	TEXT
整數 80	INTEGER	INTEGER
小數 80.5	DECIMAL 等數值型別	普通表可將小數存為 real
借出日 2026-09-01	DATE	後面的圖書館案例用固定格式 TEXT

實作額外補充（SQLite）：A、B 各從空表開始，目標皆為拒絕 80.5。

```
-- A：預期拒絕 80.5；INSERT 加入資料，SELECT 查看結果
CREATE TABLE GRADE_A (Grade INTEGER);
INSERT INTO GRADE_A VALUES (80.5);
SELECT Grade FROM GRADE_A;
```

A 實際結果：存入成功，與預期不同

SELECT 輸出 Grade = 80.5；A 表有 1 列，沒有拒絕小數。

```
-- B：CHECK 要求條件成立，預期拒絕 80.5
-- typeof 讀出儲存型別；'integer' 表示整數
CREATE TABLE GRADE_B (
  Grade INTEGER CHECK (Grade IS NULL OR typeof(Grade) = 'integer')
);
INSERT INTO GRADE_B VALUES (80.5);
```

B 實際結果：拒絕新增，符合預期

錯誤：CHECK constraint failed；B 表仍有 0 列，仍允許 NULL。

本頁重點：答案：B 達到拒絕 80.5 的目標。A 只宣告 INTEGER；B 新增整數 CHECK，才會報錯且不留下資料。

學生表的欄位與限制

第3頁的 S1、S2 都叫 Alex，學號卻不同。目標是學號唯一必填、姓名必填但允許同名；DeptId 可為 NULL。現在只有第4頁建好的系所與課程空表。

預期：下段只建立 STUDENT 的欄位與限制，學生表仍為 0 列。PRIMARY KEY 保證學號唯一；Name 不加 UNIQUE。省略 DeptId 才用 DEFAULT D1，非空代碼須參照 DEPT。

```
CREATE TABLE STUDENT (  
  StudentId TEXT NOT NULL PRIMARY KEY,  
  Name TEXT NOT NULL,  
  DeptId TEXT DEFAULT 'D1',  
  FOREIGN KEY (DeptId) REFERENCES DEPT(DeptId)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

實際結果：STUDENT 建立成功，0 列

DEPT、COURSE 也仍為 0 列。建表成功不代表 S1、S2 已經加入。

ON DELETE RESTRICT：拒絕刪除被學生參照的系所。ON UPDATE CASCADE：系所代碼更改時，同步學生的 DeptId；後面各用一次操作比較。

問題：把 Name 也加上 UNIQUE，還能保留原來兩位 Alex 嗎？

回答：不能。S1、S2 是不同學生；應讓學號唯一，不應要求所有姓名互不相同。SQLite 普通表明寫 NOT NULL，也避免部分主鍵允許 NULL 的歷史例外。

本頁重點：答案：學號用 PRIMARY KEY 與 NOT NULL；姓名只用 NOT NULL，允許同名。

選課表的主鍵、外來鍵與成績限制

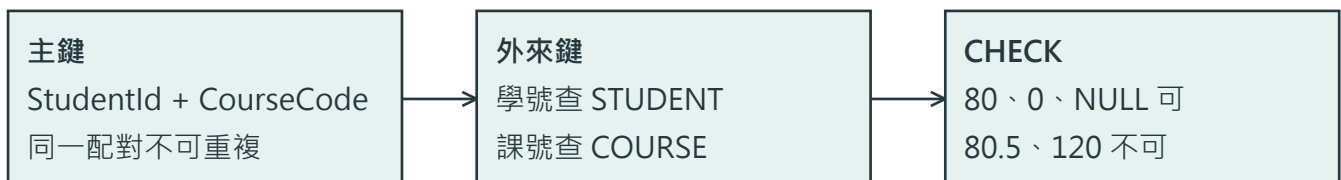
S1 可修 DB101 與 CS102；DB101 可有 S1 與 S2。目標是只禁止相同學號與課程重複。Grade 可未登錄，否則須是 0 至 100 的整數。

預期：在前三張空表已建立後，建立 ENROLLMENT 空表；兩欄合起來作主鍵，兩個代碼都必須查得到對象。Grade IS NULL 或「整數且介於 0 至 100」才通過 CHECK。

```
CREATE TABLE ENROLLMENT (
  StudentId TEXT NOT NULL REFERENCES STUDENT(StudentId),
  CourseCode TEXT NOT NULL REFERENCES COURSE(CourseCode),
  Grade INTEGER,
  PRIMARY KEY (StudentId, CourseCode),
  CONSTRAINT grade_domain CHECK (
    Grade IS NULL OR
    (typeof(Grade) = 'integer' AND Grade BETWEEN 0 AND 100)
  )
);
```

實際結果：ENROLLMENT 建立成功，0 列

四張表都仍為 0 列。上面的 SQL 宣告三種限制，尚未加入選課。



這是建表定義中各項限制的讀法，不是另外一段 INSERT 的結果。

問題：Grade 合法，是否就代表這筆新增一定成功？

回答：不是。對照第3頁：(S1,DB101,90) 的 90 合法，但 S1 / DB101 已有一筆，不能再次加入。

本頁重點：答案：成績合法不夠；配對唯一、參照存在、成績限制都要通過。

選課主鍵：學號與課程代碼

目標：同一位 S1 的 DB101 / 80、CS102 / NULL 都能存入。實作額外補充 (SQLite)
：A、B 各從空測試表開始，學生與課程使用第3頁的資料，只改主鍵。

```
-- A：預期 S1 的兩門課都能存入
CREATE TABLE WRONG_ENROLL (
  StudentId TEXT NOT NULL REFERENCES STUDENT(StudentId),
  CourseCode TEXT NOT NULL REFERENCES COURSE(CourseCode),
  Grade INTEGER CHECK (Grade IS NULL OR
    (typeof(Grade)='integer' AND Grade BETWEEN 0 AND 100)),
  PRIMARY KEY (StudentId)
);
INSERT INTO WRONG_ENROLL VALUES ('S1','DB101',80);
INSERT INTO WRONG_ENROLL VALUES ('S1','CS102',NULL);
```

A 實際結果：第二句 INSERT 報 UNIQUE 錯誤

WRONG_ENROLL 只有 S1 / DB101 / 80；第二門課被 StudentId 單欄主鍵擋住。

```
-- B：只改主鍵，仍輸入同兩筆選課
CREATE TABLE REPAIRED_ENROLL (
  StudentId TEXT NOT NULL REFERENCES STUDENT(StudentId),
  CourseCode TEXT NOT NULL REFERENCES COURSE(CourseCode),
  Grade INTEGER CHECK (Grade IS NULL OR
    (typeof(Grade)='integer' AND Grade BETWEEN 0 AND 100)),
  PRIMARY KEY (StudentId,CourseCode)
);
INSERT INTO REPAIRED_ENROLL VALUES ('S1','DB101',80);
INSERT INTO REPAIRED_ENROLL VALUES ('S1','CS102',NULL);
SELECT * FROM REPAIRED_ENROLL;
```

B 實際結果：兩筆都存入

S1 / DB101 / 80；S1 / CS102 / NULL

B 只改 PRIMARY KEY 的欄位；兩段都不更動原 ENROLLMENT。此處排好資料方便對照，SQL 未保證列序。

本頁重點：答案：B 用學號與課號合成主鍵，才允許同一學生修不同課；不要換學號來躲過 A 的錯誤。

刪除系所的外來鍵設定

D1 被 S1、S2 參照。本頁需求：若允許學生改為尚未分系，如何刪 D1 且保留三位學生與三筆選課？實作額外補充 (SQLite)：A、B 各用第3頁全部資料，只改 ON DELETE。

A 使用第6頁的完整定義：ON DELETE RESTRICT。預期刪 D1 後學生與選課仍在。

```
DELETE FROM DEPT WHERE DeptId='D1';
```

A 實際結果：FOREIGN KEY constraint failed

D1 未刪除；S1、S2 仍屬 D1。失敗當下四表全部不變，並未達到刪 D1 的目標。

B 在獨立資料庫中使用下方定義；預期學生空表建立成功。只把 RESTRICT 改為 SET NULL，其餘限制不變。

```
CREATE TABLE STUDENT (  
  StudentId TEXT NOT NULL PRIMARY KEY,  
  Name TEXT NOT NULL,  
  DeptId TEXT DEFAULT 'D1',  
  FOREIGN KEY (DeptId) REFERENCES DEPT(DeptId)  
    ON DELETE SET NULL ON UPDATE CASCADE  
);
```

B 定義的實際結果：STUDENT 建立成功，0 列

接著載入第3頁同一份資料，再執行與 A 相同的 DELETE；不是直接在 A 上重建表。

```
DELETE FROM DEPT WHERE DeptId='D1';
```

B 實際結果：D1 已刪，學生與選課保留

DEPT 只剩 D2；S1、S2、S3 的 DeptId 都是 NULL。COURSE 兩列、ENROLLMENT 三列均不變。

其他 ON DELETE：NO ACTION 本例也拒絕；CASCADE 因選課仍參照學生而失敗；SET DEFAULT 因仍填已刪的 D1 而失敗。三者四表均保留。

本頁重點：答案：在允許學生尚未分系的需求下，改為 SET NULL 才能刪 D1 且保留學生與選課。

查詢 D1 學生姓名

STUDENT 有 S1 / Alex / D1、S2 / Alex / D1、S3 / Casey / NULL。目標只列 D1 學生姓名，應有兩列 Alex，不應有 Casey。A、B 用同一份資料，只比較 WHERE。

```
-- A：先顯示每筆來源列的姓名  
SELECT Name FROM STUDENT;
```

A 實際結果：多出了未分系的 Casey
Alex ; Alex ; Casey

三列姓名都列出，不符合只列 D1 的需求。這裡只比較結果內容，不保證顯示順序。

B 預期只留下 D1 的兩列姓名：加上 WHERE DeptId='D1'，不改來源與輸出欄。

```
SELECT Name FROM STUDENT  
WHERE DeptId='D1';
```

B 實際結果：只有 D1 的兩列
Alex ; Alex

S1、S2 同名，所以有兩列 Alex。STUDENT 原本的三列都保留。

本頁重點：答案：WHERE 決定留下哪些列；SELECT Name 決定顯示哪個欄位。B 才回答 D1 的姓名。

查詢條件：AND 與 OR

三筆是 S1 / DB101 / 80、S1 / CS102 / NULL、S2 / DB101 / 0。需求是修任一門且成績至少 60，預期只有 S1 / DB101。A、B 只比較括號。

```
SELECT StudentId,CourseCode FROM ENROLLMENT
WHERE CourseCode='DB101'
      OR CourseCode='CS102' AND Grade>=60;
```

A 實際結果：零分的 S2 也被選入
S1 / DB101 ; S2 / DB101

AND 先於 OR；A 的 60 分條件只套在 CS102 那一支，DB101 可直接通過。

B 預期排除 S2 的零分：先括住兩門課的 OR，再要求 Grade>=60。

```
SELECT StudentId,CourseCode FROM ENROLLMENT
WHERE (CourseCode='DB101' OR CourseCode='CS102')
      AND Grade>=60;
```

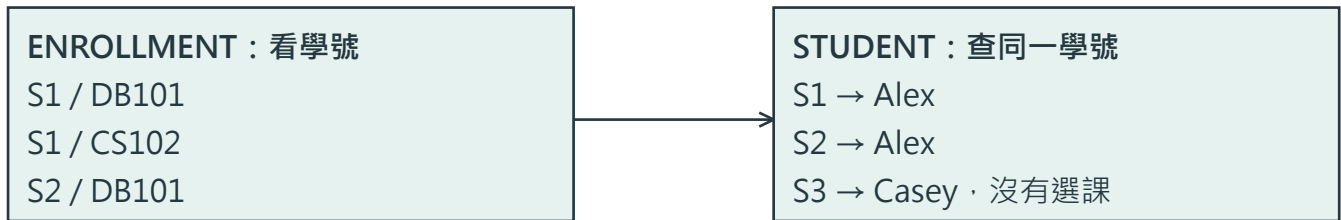
B 實際結果：只有及格的 S1
S1 / DB101

80 通過；0 不通過；未登錄的 NULL 也不會通過至少 60 的條件。

本頁重點：答案：B 的括號讓兩門課都必須檢查成績；S2 的零分直接指出 A 的問題。

查詢學生姓名與選課資料

STUDENT 有 S1 Alex、S2 Alex、S3 Casey。三筆選課存 S1 / DB101、S1 / CS102、S2 / DB101；每筆都要沿學號找到姓名。



條件：選課學號 = 學生學號；不是用 Name 配對。

S1 的兩門課都查到 S1 Alex；S2 的 DB101 查到另一位 Alex。

把兩張表的列依條件配對，稱接合 (JOIN)。兩表都有 StudentId，所以加上表名與句點。預期 S1 配到自己的兩門課，S2 配到自己的 DB101，共三列。

```
SELECT STUDENT.StudentId, STUDENT.Name,
       ENROLLMENT.CourseCode
FROM STUDENT, ENROLLMENT
WHERE STUDENT.StudentId = ENROLLMENT.StudentId;
```

實際結果：三筆配對；此處按代碼排好方便對照

StudentId	Name	CourseCode
S1	Alex	CS102
S1	Alex	DB101
S2	Alex	DB101

問題：S3 Casey 為何不在結果裡？

回答：沒有一筆選課存 S3。查詢條件只留下兩表學號相等的配對。

本頁重點：答案：三列都以相同學號接合；S3 沒有選課，不會出現在結果。

學生表的別名與自我接合

D1 有 S1 Alex、S2 Alex；S3 Casey 未分系。需求是同系的不同學生每組只列一次，預期只有 S1 / S2。a、b 是 STUDENT 兩次引用的表別名 (alias)，不是新表。

A 用 <> 排除同一學號；先猜是否也能排除 S2 / S1。FirstId、SecondId 是輸出欄的名稱。

```
-- A：預期同系的不同學生每組只列一次
SELECT a.StudentId AS FirstId,b.StudentId AS SecondId
FROM STUDENT AS a,STUDENT AS b
WHERE a.DeptId=b.DeptId AND a.StudentId <> b.StudentId;
```

A 實際結果：同一組出現兩個方向

FirstId / SecondId : S1 / S2 ; S2 / S1

<> 只要求學號不同，兩個方向都成立。

B 預期只有 S1 / S2：只把 <> 改為 <，其餘條件不變。

```
SELECT a.StudentId AS FirstId,b.StudentId AS SecondId
FROM STUDENT AS a, STUDENT AS b
WHERE a.DeptId=b.DeptId AND a.StudentId < b.StudentId;
```

B 實際結果：每組只列一次

FirstId=S1 ; SecondId=S2

兩位 Alex 的學號不同。S3 的 NULL 未通過同系相等條件，三位學生都仍保留。

本頁重點：答案：B 的 < 同時排除自己與反向重複；只用 <> 的 A 仍列出兩個方向。

漏寫配對條件的查詢結果

沿用三位學生與三筆選課；STUDENT 暫稱 s，ENROLLMENT 暫稱 e。目標仍是三筆正確配對。s.StudentId 是學生學號，e.StudentId 是選課所存學號；兩者必須相等。

A 預期也是三筆正確配對；先檢查漏掉 WHERE 會發生什麼。

```
SELECT s.StudentId,s.Name,e.CourseCode
FROM STUDENT AS s, ENROLLMENT AS e;
```

A 實際結果：3 × 3 = 9 列，不是三筆正確配對
錯誤配對之一：S3 / Casey / DB101

這裡只顯示九列中的一列反例。Casey 沒有選課，卻被列為修 DB101。

B 預期只留下三筆：補上兩個學號相等，其他資料不變。

```
SELECT s.StudentId,s.Name,e.CourseCode
FROM STUDENT AS s, ENROLLMENT AS e
WHERE s.StudentId=e.StudentId;
```

B 實際結果：只有三筆正確配對
S1 / Alex / DB101 ; S1 / Alex / CS102 ; S2 / Alex / DB101

此處把結果排好方便對照，SQL 未承諾列序；S3 沒有配對，四張來源表都不變。

問題：S3 配到 S1 / DB101，能證明 Casey 修過導論嗎？

回答：不能。那只是漏條件產生的組合；外來鍵不會自動替查詢加上配對條件。

本頁重點：答案：B 加上學號相等條件，才把選課配給正確學生；A 能執行，卻回答錯問題。

DISTINCT：去除重複輸出列

S1、S2 都叫 Alex，S3 叫 Casey。本頁需求是列出不同姓名，預期只有 Alex、Casey；不是要列出每位學生。只比較有無 DISTINCT。

```
-- A：先顯示每筆來源列的姓名  
SELECT Name FROM STUDENT;
```

A 實際結果：Alex 重複出現
Alex ; Alex ; Casey

來源有兩位 Alex。只寫 SELECT Name 不會去除重複姓名。

```
-- B：目標是不同姓名，不是不同學生  
SELECT DISTINCT Name FROM STUDENT;
```

B 實際結果：兩種不同姓名
Alex ; Casey

兩列 Alex 的輸出內容相同，只顯示一次；來源仍有 S1、S2、S3 三位學生。

問題：加 DISTINCT 後，只剩兩位學生了嗎？

回答：沒有。只顯示姓名的查詢失去了區分兩位 Alex 的學號，不是刪掉其中一個學生。

本頁重點：答案：B 回答「有哪些不同姓名」。 DISTINCT 去除重複輸出列，沒有合併或刪除學生。

兩門課學生名單的集合運算

DB101 有 S1、S2；CS102 有 S1。需求先找任一門的不同學生，預期 S1、S2。四段只換中間的運算，來源資料不變。

```
-- A：任一門的不同學生，預期 S1、S2
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='DB101'
UNION ALL SELECT StudentId FROM ENROLLMENT WHERE CourseCode='CS102';
```

A：UNION ALL 實際保留 S1 兩次

實際三列：S1、S1、S2；S1 重複，未符合不同學生的需求。

```
-- B：任一門的不同學生，預期 S1、S2
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='DB101'
UNION
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='CS102';
```

B：UNION 實際 2 列

S1、S2；去除相同輸出列，符合任一門的不同學生名單。

```
-- 改問兩門都修，預期 S1
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='DB101'
INTERSECT SELECT StudentId FROM ENROLLMENT WHERE CourseCode='CS102';
```

INTERSECT 實際 1 列

S1：只保留兩門課都有的學號。

```
-- 改問只在 DB101、不在 CS102，預期 S2
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='DB101'
EXCEPT SELECT StudentId FROM ENROLLMENT WHERE CourseCode='CS102';
```

EXCEPT 實際 1 列

S2：左邊減右邊。SQLite 不支援 INTERSECT ALL、EXCEPT ALL；此處只比較輸出內容。

本頁重點：答案：不同學生用 UNION；兩門都修用 INTERSECT；只在左邊用 EXCEPT。

集合運算的欄位對應

STUDENT 的學號是 S1、S2、S3；ENROLLMENT 是 S1、S1、S2。需求列出兩張表出現過的不同學號，預期 S1、S2、S3；每邊都應只輸出學號。

```
SELECT StudentId FROM STUDENT
UNION SELECT StudentId,CourseCode FROM ENROLLMENT;
```

A 實際結果：欄數不同，SQL 報錯

錯誤：SELECTs to the left and right of UNION do not have the same number of result columns。左邊 1 欄，右邊 2 欄，沒有產生名單。

B 預期產生三個學號：只移除右邊的 CourseCode，讓左右都是一欄 StudentId。

```
SELECT StudentId FROM STUDENT
UNION SELECT StudentId FROM ENROLLMENT;
```

B 實際結果：3 個不同學號

S1；S2；S3

兩邊的對應欄都是學號；來源學生三列、選課三列都沒有改。

本頁重點：答案：B 同時對齊欄數與意義；A 把學號和完整選課配對放在同一個位置，無法執行。

LIKE：查詢 A 開頭的姓名

S1、S2 叫 Alex，S3 叫 Casey。需求是 A 開頭的姓名，預期 S1、S2。LIKE 用模式找文字：_ 配一個字元，% 配零個以上字元。只改模式。

```
SELECT StudentId,Name FROM STUDENT
WHERE Name LIKE 'A_';
```

A 使用 A_：實際 0 列

A_ 要求 A 後只有一字元；Alex 後面有 l、e、x 三個字元，所以沒找到。

B 預期找回兩位 Alex：將 A_ 改為 A%，不限制 A 後只有一字元。

```
SELECT StudentId,Name FROM STUDENT
WHERE Name LIKE 'A%';
```

B 使用 A%：實際 2 列

S1 / Alex ; S2 / Alex

兩位 Alex 都符合 A 開頭，Casey 不符合。本例只測這些 ASCII 姓名。

本頁重點：答案：B 的 % 表示 A 後可有任意長度；A 的 _ 把姓名限制成兩個字元。

SELECT：計算成績加 5 分

原成績是 S1 / DB101 的 80、S1 / CS102 的 NULL、S2 / DB101 的 0。需求只是試算 0 至 80 分各加 5 分，預期顯示 85 與 5，不更正原成績。BETWEEN 包含兩端。

```
SELECT StudentId,CourseCode,Grade,Grade+5 AS Preview
FROM ENROLLMENT WHERE Grade BETWEEN 0 AND 80;
```

原 ENROLLMENT

S1 / DB101 / 80
S1 / CS102 / NULL
S2 / DB101 / 0

實際輸出 Grade → Preview

S1 / DB101 : 80 → 85
S2 / DB101 : 0 → 5

NULL 那列沒有通過數值範圍；0 與 80 都通過。

Preview 是查詢輸出欄的別名，不是新增到原表的欄位。

問題：查完以後，再讀 S2 / DB101 的 Grade 是多少？

回答：仍是 0。這次只有 SELECT；正式更正必須執行 UPDATE。

本頁重點：答案：SELECT 算出 Preview 的 85 與 5；原 Grade 仍是 80、NULL、0。

ORDER BY：姓名與學號排序

原學生是 S1 Alex、S2 Alex、S3 Casey。需求先按姓名升序，同名時學號降序，預期順序是 S2、S1、S3。ASC 是升序，DESC 是降序；三位都要保留。

```
-- A：預期同名時 S2 在 S1 前  
SELECT StudentId,Name FROM STUDENT ORDER BY Name ASC;
```

A 本次實際順序：S1、S2、S3
S1 / Alex ; S2 / Alex ; S3 / Casey

只按 Name，這次兩位 Alex 未依學號降序；同名者的先後本來就沒有保證。

B 預期 S2、S1、S3：只在 ORDER BY 後補上 StudentId DESC。

```
SELECT StudentId,Name FROM STUDENT  
ORDER BY Name ASC, StudentId DESC;
```

B 實際順序：S2、S1、S3
S2 / Alex ; S1 / Alex ; S3 / Casey

每個方向只套用前面的欄位；兩位 Alex 都還在，沒有因排序而去重。

本頁重點：答案：B 用第二個排序欄決定同名者的先後；A 不能保證學號降序。

INSERT：新增資料的順序

現在正式練習新增資料。A、B 都從四張已建好的空表開始，目標載入第3頁資料；本例遇錯即停。INSERT INTO 指定目的表，VALUES 填入各欄；NULL 不加引號。

```
-- A：預期載入第3頁資料；但四張表目前全空
INSERT INTO STUDENT(StudentId,Name,DeptId) VALUES
  ('S1','Alex','D1'),('S2','Alex','D1'),('S3','Casey',NULL);
INSERT INTO DEPT VALUES ('D1','資訊管理'),('D2','會計');
INSERT INTO COURSE VALUES ('DB101'),('CS102');
INSERT INTO ENROLLMENT VALUES
  ('S1','DB101',80),('S1','CS102',NULL),('S2','DB101',0);
```

A 實際結果：FOREIGN KEY constraint failed

首句因 D1 不存在而失敗；後面的 INSERT 未執行，所以四表仍空，不是整段自動回滾。

B 預期先有系所與課程，再加入學生：在另一份空資料庫依下方順序載入。

```
INSERT INTO DEPT VALUES ('D1','資訊管理'),('D2','會計');
INSERT INTO COURSE VALUES ('DB101'),('CS102');
INSERT INTO STUDENT(StudentId,Name,DeptId) VALUES
  ('S1','Alex','D1'),('S2','Alex','D1'),
  ('S3','Casey',NULL);
```

B 第一段實際結果：系所 2、課程 2、學生 3 列

D1、D2；DB101、CS102；S1、S2、S3 已載入。選課仍為 0 列。

接著預期三筆選課都成功：現在學號與課程代碼已查得到。

```
INSERT INTO ENROLLMENT(StudentId,CourseCode,Grade) VALUES
  ('S1','DB101',80), ('S1','CS102',NULL), ('S2','DB101',0);
```

B 第二段實際結果：ENROLLMENT 3 列

S1 / DB101 / 80、S1 / CS102 / NULL、S2 / DB101 / 0。四表內容完整等於第3頁。

本頁重點：答案：先載入被參照的系所、學生與課程；B 的順序成功，A 在 D1 不存在時被拒絕。

DEFAULT 與 NULL

Dana 的學號是 S4，尚未分系。需求是新增 S4 且 DeptId 為 NULL。A、B 各從原三位學生開始；省略 DeptId 會用 DEFAULT D1，明填 NULL 才表示未分系。

```
INSERT INTO STUDENT(StudentId,Name) VALUES ('S4','Dana');  
SELECT StudentId,Name,DeptId FROM STUDENT  
WHERE StudentId='S4';
```

A 省略 DeptId：實際被填入 D1
S4 / Dana / D1

沒有達到「尚未分系」的目標，因為 DEFAULT 在省略欄位時生效。

B 預期 S4 / Dana / NULL：只改為明確填入 NULL，使用另一份沒有 S4 的原資料。

```
INSERT INTO STUDENT VALUES ('S4','Dana',NULL);  
SELECT StudentId,Name,DeptId FROM STUDENT  
WHERE StudentId='S4';
```

B 明填 NULL：實際仍是 NULL
S4 / Dana / NULL

B 不使用 DEFAULT；原 S1、S2、S3 及另外三張表都保留。

問題：可以把 S3 原有的 NULL 也解讀成 D1 嗎？

回答：不可以。S3 明確是尚未分系；DEFAULT 不會事後替所有 NULL 補上 D1。

本頁重點：答案：B 明填 NULL 才記錄尚未分系；A 省略欄位，會套用預設 D1。

INSERT SELECT：加入查詢結果

目的表 STUDENT_LIST 以學號為主鍵，目前 0 列。來源三筆選課的學號是 S1、S1、S2。需求存一份不重複名單，預期 S1、S2；INSERT SELECT 把查詢結果加入表。

```
CREATE TABLE STUDENT_LIST (  
    StudentId TEXT NOT NULL PRIMARY KEY  
);  
INSERT INTO STUDENT_LIST  
SELECT StudentId FROM ENROLLMENT;
```

A 實際結果：UNIQUE constraint failed

第二次加入 S1 違反主鍵；這句 INSERT 的資料全部撤銷，目的表仍為 0 列，表沒有消失。

B 接在同一連線的失敗狀態：預期存入 S1、S2，只在查詢增加 DISTINCT。

```
INSERT INTO STUDENT_LIST  
SELECT DISTINCT StudentId FROM ENROLLMENT;  
SELECT StudentId FROM STUDENT_LIST ORDER BY StudentId;
```

B 實際結果：STUDENT_LIST 2 列 S1 ; S2

三筆來源選課都保留；只去除輸出中的重複 S1，再加入目的表。

本頁重點：答案：B 用 DISTINCT 避免重複學號；來自 SELECT 的資料也必須滿足目的表主鍵。

CREATE TABLE AS：建立成績副本

S2 / DB101 原成績為 0。教務處先保存三筆成績，再核准將原表這一筆改為 5。目標是查到更正後正式成績 5，不是查舊副本；本頁操作依序進行。

共同準備：CREATE TABLE ... AS SELECT 建立副本；預期先留下三筆原成績，再只更正原表的 S2。

```
CREATE TABLE ENROLL_COPY AS SELECT * FROM ENROLLMENT;
UPDATE ENROLLMENT SET Grade=5
WHERE StudentId='S2' AND CourseCode='DB101';
```

準備完成：原表與副本各有 3 列

ENROLLMENT 的 S2 / DB101 = 5；ENROLL_COPY 的同筆 = 0

另外兩筆仍是 80、NULL；其餘三張表不變。副本保留建立當下的資料。

```
-- A：預期確認更正後的正式成績 5
SELECT Grade FROM ENROLL_COPY
WHERE StudentId='S2' AND CourseCode='DB101';
```

A 實際結果：Grade = 0

查到副本中的舊值，沒有回答更正後的正式成績。

```
-- B：只改查詢來源，預期得到正式成績 5
SELECT Grade FROM ENROLLMENT
WHERE StudentId='S2' AND CourseCode='DB101';
```

B 實際結果：Grade = 5

A、B 讀的是同一個更正後狀態，只改 FROM 的表名；查詢本身不再更正資料。

補充：SQLite 這種副本只複製內容，不會帶入原表的主鍵與其他限制。

本頁重點：答案：CREATE TABLE AS 的副本不自動同步；B 查 ENROLLMENT 才查到目前正式成績 5。

DELETE：刪除誤登選課

核准只移除誤登的 S1 / CS102；S1 / DB101 的 80、S2 / DB101 的 0 都要保留。A、B 各重設第3頁資料，學生名單也不能刪。只比較 WHERE 是否寫完整配對。

```
-- A：預期只移除 S1 的 CS102
DELETE FROM ENROLLMENT WHERE StudentId='S1';
SELECT * FROM ENROLLMENT ORDER BY StudentId,CourseCode;
```

A 實際結果：S1 的兩筆選課都刪了
剩下：S2 / DB101 / 0

原本正確的 S1 / DB101 / 80 也消失。學生三列仍在，但選課刪錯了。

B 預期只刪一列、保留另外兩列：先查目標，再用相同的完整配對條件 DELETE。

```
SELECT * FROM ENROLLMENT
WHERE StudentId='S1' AND CourseCode='CS102';
DELETE FROM ENROLLMENT
WHERE StudentId='S1' AND CourseCode='CS102';
SELECT * FROM ENROLLMENT ORDER BY StudentId,CourseCode;
```

B 實際結果：只移除誤登，仍有 2 列
S1 / DB101 / 80；S2 / DB101 / 0

第一個 SELECT 找到 S1 / CS102 / NULL；最後一個 SELECT 確認其他兩筆保留。其餘三張表全部不變。

本頁重點：答案：B 同時指定學號與課號，才只刪核准的一筆；A 只限 S1，會刪掉他的兩門課。

UPDATE：更正指定選課的成績

核准只把 S1 / DB101 的 80 更正為 85。預期 S1 / CS102 仍為 NULL，S2 / DB101 仍為 0。A、B 各重設第3頁資料，只比較 WHERE 是否包含完整配對。

```
-- A：預期只把 S1 的 DB101 改成 85
UPDATE ENROLLMENT SET Grade=85 WHERE StudentId='S1';
SELECT * FROM ENROLLMENT ORDER BY StudentId,CourseCode;
```

A 實際結果：S1 的兩門課都改成 85

S1 / CS102 / 85 ; S1 / DB101 / 85 ; S2 / DB101 / 0

只限 StudentId=S1，連尚未登錄的 CS102 也被填了成績。85 合法，仍可能改錯列。

B 預期只改一列：先查完整學號與課號，再用相同條件 UPDATE。

```
SELECT * FROM ENROLLMENT
WHERE StudentId='S1' AND CourseCode='DB101';
UPDATE ENROLLMENT SET Grade=85
WHERE StudentId='S1' AND CourseCode='DB101';
SELECT * FROM ENROLLMENT ORDER BY StudentId,CourseCode;
```

B 實際結果：只更正 DB101，其他兩筆保留

S1 / CS102 / NULL ; S1 / DB101 / 85 ; S2 / DB101 / 0

第一個 SELECT 查到 S1 / DB101 / 80；最後一個 SELECT 顯示完整三列。其餘三張表不變。

本頁重點：答案：B 的 WHERE 同時指定學號與課號，才只更正核准的一筆；A 誤改 CS102 的 NULL。

UPDATE：更改系所代碼

獨立重設第3頁的全部資料：S1、S2 屬 D1，S3 未分系。需求只把資管系代碼 D1 改為 D10，學生、課程與成績都要保留。ON UPDATE CASCADE 已在第6頁宣告。

預期：D1 改為 D10，S1、S2 的 DeptId 同步變 D10；S3 的 NULL 不變。

```
UPDATE DEPT SET DeptId='D10' WHERE DeptId='D1';  
SELECT StudentId,DeptId FROM STUDENT ORDER BY StudentId;
```

更改前

DEPT：D1 資訊管理
S1 → D1；S2 → D1
S3 → NULL

更改後

DEPT：D10 資訊管理
S1 → D10；S2 → D10
S3 → NULL

系所代碼的變更傳到學生的外來鍵，不改學生學號，也不改三筆選課。

問題：ON DELETE SET NULL 會控制這次更改嗎？

回答：不會。這次執行 UPDATE，不是 DELETE；要看 ON UPDATE 的設定。

本頁重點：答案：這次是 UPDATE，ON UPDATE CASCADE 同步兩位學生的系所代碼；四表的其他事實保留。

圖書館案例：會員、實體書與借閱

圖書館記錄哪位會員在何日借走哪一本實體書，以及何日歸還。以下全部是固定人工教學資料，不是真實讀者紀錄。



箭頭：借閱列依代碼找到既有會員與實體書。

本例只有會員、實體書、借閱三種紀錄；書名相同也不代表同一本實體書。

MemberId、CopyId 分別識別會員與實體書；LoanId 是全館唯一的每次借閱代碼。同一人先還再借同一本書，要新增借閱列，不覆寫舊紀錄。

每筆借閱必填 LoanId、MemberId、CopyId、BorrowedOn；會員與書都必須已存在。ReturnedOn 可為 NULL；已填時不能早於借出日。Email 可缺，但已知值不能是空字串，也不能與別人重複。

同一本實體書不能同時借給兩人。日期先確認有效，統一寫成 YYYY-MM-DD，例如 2026-09-01；本章的簡單 DDL 不檢查全部跨列規則。

本頁重點：先辨認人、實體書與每次借閱；業務規則不等於已全部寫成資料庫限制。

圖書館案例：完整範例資料

以下列出全部起始資料，日期均為 2026 年。NULL 表示未記錄 Email 或尚未歸還；修改各自重設。

MEMBER：會員

MemberId	Name	Email
M1	小安	NULL
M2	小文	wen@lib.test

COPY：實體書

CopyId	Title
B1	SQL入門
B2	SQL_練習

LOAN：每次借閱

LoanId	MemberId	CopyId	BorrowedOn	ReturnedOn
L1	M1	B1	2026-09-01	2026-09-05
L2	M1	B1	2026-09-10	2026-09-15
L3	M2	B2	2026-09-11	NULL



追蹤 2026 年的 L1、L2：人和書相同，日期分開；兩筆借閱都要留下。

問題：沿兩條線，誰先還再借同一本？

回答：M1 小安借 B1；L1 已還，才借 L2。兩段不重疊。

本頁重點：L1 與 L2 是同人同書的兩次借閱；L3 是小文借 B2，尚未歸還。

會員表的 Email 限制

需求：小安 M1 尚未提供 Email，仍可加入會員。預期省略 Email 後存入 NULL。實作額外補充 (SQLite)：兩段各從空表開始，只改 Email 是否必填。

```
CREATE TABLE MEMBER (  
  MemberId TEXT NOT NULL PRIMARY KEY, Name TEXT NOT NULL,  
  Email TEXT NOT NULL UNIQUE CHECK (Email IS NULL OR Email <> '')  
);  
INSERT INTO MEMBER(MemberId, Name) VALUES ('M1', '小安');  
SELECT MemberId, Name, Email FROM MEMBER;
```

第一段實際：新增失敗

NOT NULL constraint failed: MEMBER.Email

MEMBER 仍為空表；不符合「未提供 Email 也能加入」的需求。

```
CREATE TABLE MEMBER (  
  MemberId TEXT NOT NULL PRIMARY KEY, Name TEXT NOT NULL,  
  Email TEXT UNIQUE CHECK (Email IS NULL OR Email <> '')  
);  
INSERT INTO MEMBER(MemberId, Name) VALUES ('M1', '小安');  
SELECT MemberId, Name, Email FROM MEMBER;
```

第二段實際：新增成功

MemberId = M1 ; Name = 小安 ; Email = NULL

移除 Email 的 NOT NULL；未設 DEFAULT，所以省略時存 NULL。

本頁重點：答案：Email 可缺，移除 NOT NULL；UNIQUE 保留已知值不重複，CHECK 保留不可空字串。

借閱表的主鍵、外來鍵與日期限制

需求：L3 是小文 M2 借 B2，09-11 借出，核對後應登記 09-16 歸還。預期新增成功。
實作額外補充 (SQLite)：會員、書及外鍵檢查已備妥；兩段各建同樣的空借閱表，只改歸還日。

```
CREATE TABLE LOAN (  
  LoanId TEXT NOT NULL PRIMARY KEY,  
  MemberId TEXT NOT NULL REFERENCES MEMBER(MemberId)  
    ON DELETE RESTRICT,  
  CopyId TEXT NOT NULL REFERENCES COPY(CopyId),  
  BorrowedOn TEXT NOT NULL, ReturnedOn TEXT,  
  CHECK (ReturnedOn IS NULL OR ReturnedOn >= BorrowedOn)  
);  
INSERT INTO LOAN VALUES  
  ('L3', 'M2', 'B2', '2026-09-11', '2026-09-09');
```

第一段實際：09-09 被拒絕

CHECK constraint failed ; LOAN 仍為空表

09-09 早於借出日 09-11，不能登記這筆還書。

```
CREATE TABLE LOAN (  
  LoanId TEXT NOT NULL PRIMARY KEY,  
  MemberId TEXT NOT NULL REFERENCES MEMBER(MemberId)  
    ON DELETE RESTRICT,  
  CopyId TEXT NOT NULL REFERENCES COPY(CopyId),  
  BorrowedOn TEXT NOT NULL, ReturnedOn TEXT,  
  CHECK (ReturnedOn IS NULL OR ReturnedOn >= BorrowedOn)  
);  
INSERT INTO LOAN VALUES  
  ('L3', 'M2', 'B2', '2026-09-11', '2026-09-16');  
SELECT LoanId, BorrowedOn, ReturnedOn FROM LOAN;
```

第二段實際：09-16 新增成功

L3 | BorrowedOn 2026-09-11 | ReturnedOn 2026-09-16

使用已核對的 09-16；這次新建空表，不延續第一段的執行狀態。

本頁重點：答案：CHECK 正確且未改；錯的是第一段的 09-09。改填已核對的 09-16，即可登記。

查詢借閱人與書名

需求：每次借閱只列真正的借閱人與書名，預期三列。L1、L2 是小安借 B1；L3 是小文借 B2。L、M、C 暫稱借閱、會員、實體書表；只比較書的配對條件。

```
SELECT L.LoanId, M.Name, C.Title
FROM LOAN AS L, MEMBER AS M, COPY AS C
WHERE L.MemberId = M.MemberId
ORDER BY L.LoanId, C.CopyId;
```

第一段實際：6 列，不符合預期

示例：L1 | 小安 | SQL 入門；L1 | 小安 | SQL_練習

L2、L3 也各配上兩本書；不是六筆真實借閱。

```
SELECT L.LoanId, M.Name, C.Title
FROM LOAN AS L, MEMBER AS M, COPY AS C
WHERE L.MemberId = M.MemberId AND L.CopyId = C.CopyId
ORDER BY L.LoanId, C.CopyId;
```

第二段實際：3 列（完整輸出）

L1 | 小安 | SQL 入門
L2 | 小安 | SQL 入門
L3 | 小文 | SQL_練習

補上 L.CopyId = C.CopyId，只保留每次真正借的書。

本頁重點：答案：只配會員還不夠；借閱的 CopyId 也必須配到實體書的 CopyId。

DISTINCT：查詢不重複的借閱會員

需求：曾借過書的會員各列一次，預期只有 M1、M2。原資料是 L1→M1、L2→M1、L3→M2。兩段都有 DISTINCT，只改輸出欄位及對應的排序欄。

```
SELECT DISTINCT MemberId, LoanId FROM LOAN  
ORDER BY MemberId, LoanId;
```

第一段實際：3 列 (完整輸出)

M1 | L1 ; M1 | L2 ; M2 | L3

仍有兩列 M1，因為兩列的 LoanId 不同。

```
SELECT DISTINCT MemberId FROM LOAN ORDER BY MemberId;
```

第二段實際：2 列 (完整輸出)

M1 ; M2

只輸出 MemberId，兩次 M1 才是相同的完整輸出列。

本頁重點：答案：DISTINCT 比較全部輸出欄位；要會員各一次，就不要同時輸出 LoanId。

借閱會員名單的集合運算

需求：合併曾借 B1 或 B2 的會員名單，每人只列一次，預期 M1、M2。B1 的兩筆借閱輸出 M1、M1；B2 輸出 M2。兩段的 SELECT 完全相同，只換合併運算子。

```
SELECT MemberId FROM LOAN WHERE CopyId = 'B1'  
UNION ALL  
SELECT MemberId FROM LOAN WHERE CopyId = 'B2'  
ORDER BY MemberId;
```

第一段實際：3 列 (完整輸出)

M1 ; M1 ; M2

UNION ALL 保留兩次 M1，不符合每人一次的需求。

```
SELECT MemberId FROM LOAN WHERE CopyId = 'B1'  
UNION  
SELECT MemberId FROM LOAN WHERE CopyId = 'B2'  
ORDER BY MemberId;
```

第二段實際：2 列 (完整輸出)

M1 ; M2

UNION 合併後去除相同的完整輸出列。

本頁重點：答案：名單需去重就用 UNION；需保留原重複次數才用 UNION ALL。

LIKE：比對書名中的底線

需求：找書名真的以 SQL_ 開頭的書，預期只列 B2。原資料是 B1「SQL入門」、B2「SQL練習」。LIKE 的 _ 原本配一個任意字元；兩段只改底線的比對寫法。

```
SELECT CopyId, Title FROM COPY
WHERE Title LIKE 'SQL_%' ORDER BY CopyId;
```

第一段實際：B1、B2 都選到（完整輸出）

B1 | SQL入門；B2 | SQL練習

B1 的「入」也配到模式中的 _，因此多選一行。

修正：以 ESCAPE '!' 指定標記；!_ 要求書名中的底線本身。

```
SELECT CopyId, Title FROM COPY
WHERE Title LIKE 'SQL!_%' ESCAPE '!' ORDER BY CopyId;
```

第二段實際：只選 B2（完整輸出）

B2 | SQL練習

其他資料與排序不變；% 仍配零個以上字元。

本頁重點：答案：要找字面底線，用 !_ 加 ESCAPE，而不是直接把 _ 當普通字元。

BETWEEN：查詢借出日期範圍

需求：列出 09-01 至 09-10 借出的紀錄，包含首尾兩天，預期 L1、L2。三筆借出日是 L1=09-01、L2=09-10、L3=09-11；均為已確認有效的 2026 年固定格式日期。

```
SELECT LoanId, BorrowedOn FROM LOAN
WHERE BorrowedOn > '2026-09-01' AND BorrowedOn < '2026-09-10'
ORDER BY BorrowedOn, LoanId;
```

第一段實際：零列

> 與 < 排除首尾兩天；L1、L2 都在端點，因此都沒選到。

```
SELECT LoanId, BorrowedOn FROM LOAN
WHERE BorrowedOn BETWEEN '2026-09-01' AND '2026-09-10'
ORDER BY BorrowedOn, LoanId;
```

第二段實際：2 列（完整輸出）

L1 | 2026-09-01 ; L2 | 2026-09-10

BETWEEN 含兩個端點；L3 超過上界，仍不選入。

本頁重點：答案：本需求包含首尾兩天，使用 BETWEEN；嚴格 >、< 不符合需求。

INSERT：補登借閱紀錄

需求：補登小安 M1 在 08-10 借 B1、08-15 歸還的一次借閱，預期增加一列。L1–L3 已存在；兩段各重新載入原資料，只改新紀錄的 LoanId。

```
INSERT INTO LOAN
  (LoanId, MemberId, CopyId, BorrowedOn, ReturnedOn)
VALUES ('L1', 'M1', 'B1', '2026-08-10', '2026-08-15');
SELECT LoanId, MemberId, CopyId FROM LOAN
WHERE CopyId = 'B1' ORDER BY BorrowedOn, LoanId;
```

第一段實際：沿用 L1，新增失敗

UNIQUE constraint failed: LOAN.LoanId

原 LOAN 仍只有 L1、L2、L3；八月借閱沒有存入。

```
INSERT INTO LOAN
  (LoanId, MemberId, CopyId, BorrowedOn, ReturnedOn)
VALUES ('L4', 'M1', 'B1', '2026-08-10', '2026-08-15');
SELECT LoanId, MemberId, CopyId FROM LOAN
WHERE CopyId = 'B1' ORDER BY BorrowedOn, LoanId;
```

第二段實際：新代碼 L4，新增成功

B1 查詢完整輸出：L4 | M1 | B1；L1 | M1 | B1；L2 | M1 | B1

全表四列；L4 是八月紀錄，所以依 BorrowedOn 排在 L1 前。會員及書不變。

本頁重點：答案：新的一次借閱使用未出現的 LoanId；新增 L4，不覆寫舊 L1。

CREATE TABLE AS：建立借閱副本

需求：複製 L1–L3，副本的 LoanId 也不可重複。預期再載入 L3 時拒絕、仍三列。實作額外補充（SQLite）：兩段各重新載入原資料，只改副本的建表方式。

```
CREATE TABLE LOAN_SNAPSHOT AS SELECT * FROM LOAN;  
INSERT INTO LOAN_SNAPSHOT  
SELECT * FROM LOAN WHERE LoanId = 'L3';  
SELECT LoanId, ReturnedOn FROM LOAN_SNAPSHOT ORDER BY LoanId;
```

第一段實際：副本 4 列（完整輸出）

L1 | 09-05 ; L2 | 09-15 ; L3 | NULL ; L3 | NULL

日期均為 2026 年。CREATE TABLE AS 沒有複製主鍵，重複 L3 仍存入。

```
CREATE TABLE LOAN_SNAPSHOT (  
  LoanId TEXT NOT NULL PRIMARY KEY,  
  MemberId TEXT NOT NULL REFERENCES MEMBER(MemberId)  
    ON DELETE RESTRICT,  
  CopyId TEXT NOT NULL REFERENCES COPY(CopyId),  
  BorrowedOn TEXT NOT NULL, ReturnedOn TEXT,  
  CHECK (ReturnedOn IS NULL OR ReturnedOn >= BorrowedOn)  
);  
INSERT INTO LOAN_SNAPSHOT SELECT * FROM LOAN;  
INSERT INTO LOAN_SNAPSHOT  
SELECT * FROM LOAN WHERE LoanId = 'L3';
```

第二段實際：重複 L3 被拒絕

UNIQUE constraint failed: LOAN_SNAPSHOT.LoanId

副本仍為 L1=09-05、L2=09-15、L3=NULL；原 LOAN、MEMBER、COPY 均不變。

本頁重點：答案：需要副本的限制，就先明寫 CREATE TABLE 的限制，再 INSERT SELECT 載入資料。

刪除會員的外來鍵設定

需求：保留借閱歷史及借閱人，預期拒絕刪除仍被 L1、L2 參照的 M1。兩段各從相同會員、書且尚無借閱表開始，載入相同三列，只改 ON DELETE 設定。

```
CREATE TABLE LOAN (  
  LoanId TEXT NOT NULL PRIMARY KEY, MemberId TEXT NOT NULL  
    REFERENCES MEMBER(MemberId) ON DELETE CASCADE,  
  CopyId TEXT NOT NULL REFERENCES COPY(CopyId),  
  BorrowedOn TEXT NOT NULL, ReturnedOn TEXT,  
  CHECK (ReturnedOn IS NULL OR ReturnedOn >= BorrowedOn));  
INSERT INTO LOAN VALUES  
  ('L1', 'M1', 'B1', '2026-09-01', '2026-09-05'),  
  ('L2', 'M1', 'B1', '2026-09-10', '2026-09-15'),  
  ('L3', 'M2', 'B2', '2026-09-11', NULL);  
DELETE FROM MEMBER WHERE MemberId = 'M1';
```

第一段實際：M1、L1、L2 一起刪除

MEMBER 剩 M2；LOAN 剩 L3；COPY 仍有 B1、B2。歷史被刪，不符合需求。

```
CREATE TABLE LOAN (  
  LoanId TEXT NOT NULL PRIMARY KEY, MemberId TEXT NOT NULL  
    REFERENCES MEMBER(MemberId) ON DELETE RESTRICT,  
  CopyId TEXT NOT NULL REFERENCES COPY(CopyId),  
  BorrowedOn TEXT NOT NULL, ReturnedOn TEXT,  
  CHECK (ReturnedOn IS NULL OR ReturnedOn >= BorrowedOn));  
INSERT INTO LOAN VALUES  
  ('L1', 'M1', 'B1', '2026-09-01', '2026-09-05'),  
  ('L2', 'M1', 'B1', '2026-09-10', '2026-09-15'),  
  ('L3', 'M2', 'B2', '2026-09-11', NULL);  
DELETE FROM MEMBER WHERE MemberId = 'M1';
```

第二段實際：FOREIGN KEY constraint failed

MEMBER 仍有 M1、M2；LOAN 仍有 L1、L2、L3；COPY 不變。

本頁重點：答案：本需求使用 RESTRICT 拒絕刪除；CASCADE 會連參照它的借閱一起刪。

DELETE：刪除誤登借閱

獨立情境：館員核對 L3 是誤登，現在只撤銷它，預期保留 L1、L2。不是因為尚未還書就刪除。兩段各重新載入原資料，只改 DELETE 的 WHERE。

```
DELETE FROM LOAN;  
SELECT LoanId FROM LOAN ORDER BY LoanId;
```

第一段實際：LOAN 零列

三筆借閱全被刪，表仍存在；MEMBER、COPY 均不變。

```
DELETE FROM LOAN WHERE LoanId = 'L3';  
SELECT LoanId FROM LOAN ORDER BY LoanId;
```

第二段實際：留下 2 列（完整查詢輸出）

L1 ; L2

只刪 L3；小文 M2 與 B2 仍在各自的表中。

本頁重點：答案：WHERE LoanId = 'L3' 才只撤銷指定借閱；沒有 WHERE 就刪除整張表的資料列。

UPDATE：登記指定借閱的還書日期

需求：館員核對只有 L3 於 09-16 歸還；L1=09-05、L2=09-15 不改，L3 原為 NULL。
預期只改一行。兩段各重新載入原資料，同填 2026-09-16，只改 WHERE 範圍。

```
UPDATE LOAN SET ReturnedOn = '2026-09-16';  
SELECT LoanId, ReturnedOn FROM LOAN ORDER BY LoanId;
```

第一段實際：3 列都被更新（完整查詢輸出）

L1 | 2026-09-16 ; L2 | 2026-09-16 ; L3 | 2026-09-16

沒有 WHERE，連 L1、L2 的原歸還日也被覆寫。

```
UPDATE LOAN SET ReturnedOn = '2026-09-16'  
WHERE LoanId = 'L3';  
SELECT LoanId, ReturnedOn FROM LOAN ORDER BY LoanId;
```

第二段實際：只更新 L3（完整查詢輸出）

L1 | 2026-09-05 ; L2 | 2026-09-15 ; L3 | 2026-09-16

只把 L3 的 NULL 改為 09-16；會員、書及其他欄位不變。

本頁重點：答案：日期通過 CHECK 不代表修改範圍正確；WHERE LoanId = 'L3' 才限定這一次。